



4η Εργαστηριακή Άσκηση

Ψηφιακά Συστήματα VLSI

Παναγιώτης Γερασιμόπουλος 03115208
personal@devcol.com

1. FIR

Έγινε μια μικρή αλλαγή στο Control Unit του φίλτρου για να μπορεί να υποστηρίξει stall οποιασδήποτε διάρκειας στο ValidIn. Ακόμα δεν υποστηρίζει να έρθει είσοδος πιο νωρίς αλλά μπορεί να αργήσει περισσότερο από 7 κύκλους χωρίς πρόβλημα.

```
1 library ieee;
2 use ieee.std_logic_1164.all;
3 use ieee.std_logic_unsigned.all;
4
5 entity CU is
6   Port(
7     Clk, Reset, ValidIn      : in std_logic;
8     ValidOut, RomEnable, RamEnable : out std_logic;
9     RamWEnable, MacEnable, MacInit : out std_logic;
10    RamAddress, RomAddress     : out std_logic_vector(2 downto 0)
11  );
12 end entity;
13
14 architecture Behavioral of CU is
15   signal Counter : std_logic_vector(2 downto 0) := (others => '0');
16   -- NOTE(acol): Added busy flag
17   signal Busy : std_logic := '0'
18 begin
19   process(Clk, Reset) begin
20     if Reset = '1' then
21       Counter    <= (others => '0');
22       ValidOut    <= '0';
23       RamEnable   <= '0';
24       RamWEnable  <= '0';
25       MacEnable   <= '0';
26       MacInit     <= '0';
27
28     elsif rising_edge(Clk) then
29       RomEnable   <= '1';
30       RamEnable   <= '1';
31       MacEnable   <= '1';
32       MacInit     <= '0';
33       ValidOut    <= '0';
34       RamWEnable  <= '0';
35
36       -- NOTE(acol): Only send ValidOut if last 8 cycles where after a new input
37       if Counter = "111" and Busy = '1' then
38         ValidOut <= '1';
39         Busy     <= '0';
40       end if;
41
42       -- NOTE(acol): Busy = 1 after useful input
43       if ValidIn = '1' then
44         RamWEnable <= '1';
45         MacInit    <= '1';
46         Counter    <= "000";
47         Busy       <= '1';
48
49       -- NOTE(acol): Dont count if not calculating something,
50       --               not needed but maybe less energy use?
51       elsif Busy = '1' then
52         Counter <= Counter + 1;
53       end if;
54     end if;
55   end process;
56   RamAddress <= Counter;
57   RomAddress <= Counter;
58
59 end architecture;
60
```

2. Axi Lite

Για την σύνδεση του φίλτρου με το harness του axi lite καταρχάς καθυστερούμε σε ένα register για έναν κύκλο το write flag. Οπότε παίρνουμε τα δεδομένα που γράφηκαν στο register A τον επόμενο κύκλο και κάνουμε το write and με το ValidIn για να μην διαβάζει το φίλτρο ValidIn κάθε κύκλο μέχρι να ξαναγράψουμε 0 στο register.

Επίσης κάθε φορά που έχουμε νέα έξοδο την κρατάμε σε ένα register μέχρι να διαβαστεί από την C όπου και κάνουμε το ValidOut bit της 0.

```
1 process(S_AXI_ACLK) begin
2   if rising_edge(S_AXI_ACLK) then
3     if S_AXI_ARESETN = '0' then
4       WrenSkew <= '0';
5       FirValidIn <= '0';
6     else
7       -- NOTE(acol): hold write event to A register for 1 cycle :)
8       if slv_reg_wren = '1' and
9         axi_awaddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB) = b"00" then
10        WrenSkew <= '1';
11      else
12        WrenSkew <= '0';
13      end if;
14    end if;
15    FirValidIn <= WrenSkew and slv_reg0(8);
16  end if;
17 end process;
18
19 -- NOTE(acol): capture validout values
20 process(S_AXI_ACLK) begin
21   if rising_edge(S_AXI_ACLK) then
22     if S_AXI_ARESETN = '0' then
23       CaptureY <= (others => '0');
24       DataReady <= '0';
25     else
26       -- save valid
27       if FirValidOut = '1' then
28         CaptureY <= FirY;
29         DataReady <= '1';
30       end if;
31       -- clear on read
32       if slv_reg_rden = '1' and
33         axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB) = b"01" then
34         DataReady <= '0';
35       end if;
36     end if;
37   end if;
38 end process;
```

Παρακάτω είναι και το boilerplate.

```
1 -- NOTE(acol): Declarations
2 component FirFilter is
3   Port(
4     Clk      : in std_logic;
5     Reset    : in std_logic;
6     ValidIn  : in std_logic;
7     X        : in std_logic_vector(7 downto 0);
8     ValidOut : out std_logic;
9     Y        : out std_logic_vector(18 downto 0)
10  );
11 end component FirFilter;
12
```

```

13 signal FirValidIn, FirReset, FirValidOut : std_logic;
14 signal FirX : std_logic_vector(7 downto 0);
15 signal FirY : std_logic_vector(18 downto 0);
16
17 signal CaptureY : std_logic_vector(18 downto 0) := (others => '0');
18 signal DataReady : std_logic := '0';
19 signal WrenSkew : std_logic := '0';
20 begin
21
22 -- ***** --
23 -- NOTE(acol): Read
24
25 FIR : FirFilter port map(
26   Clk => S_AXI_ACLK,
27   Reset => FirReset,
28   ValidIn => FirValidIn,
29   ValidOut => FirValidOut,
30   X => FirX,
31   Y => FirY
32 );
33
34 FirReset <= slv_reg0(9);
35 FirX <= slv_reg0(7 downto 0);
36
37 -- ***** --
38
39
40 process (DataReady ,slv_reg0, slv_reg1, slv_reg2, slv_reg3,
41         axi_araddr, S_AXI_ARESETN, slv_reg_rden)
42 variable loc_addr :std_logic_vector(OPT_MEM_ADDR_BITS downto 0);
43 begin
44   -- Address decoding for reading registers
45   loc_addr := axi_araddr(ADDR_LSB + OPT_MEM_ADDR_BITS downto ADDR_LSB);
46   case loc_addr is
47     when b"00" =>
48       reg_data_out <= slv_reg0;
49     when b"01" =>
50       reg_data_out(18 downto 0) <= CaptureY;
51       reg_data_out(19) <= DataReady;
52       reg_data_out(31 downto 20) <= (others => '0');
53     when b"10" =>
54       reg_data_out <= slv_reg2;
55     when b"11" =>
56       reg_data_out <= slv_reg3;
57     when others =>
58       reg_data_out <= (others => '0');
59   end case;
60 end process;

```

3. C

Το μονό ίσως άξιο σχολιασμού κομμάτι του κώδικα είναι ότι κάνουμε loop για το μήκος του σήματος + το μήκος του impulse response - 1 και στην γραμμή 47 παίρνουμε σαν δεδομένα το σήμα για όλες τις τιμές του και μετά μηδενικά.

```

1 /*
2 // NOTE(acol): most sane path to place all the useful macros and functions
3 design_1_wrapper/ps7_cortexa9_0/standalone_ps7_cortexa9_0/bsp/ps7_cortexa9_0/include/
4
5 #define XPAR_FIR_AXI_LITE_0_DEVICE_ID 0
6 #define BASEADDR 0x43C00000
7 #define XPAR_FIR_AXI_LITE_0_S00_AXI_HIGHADDR 0x43C0FFFF
8 */
9

```

```

10 #include <stdio.h>
11 #include "platform.h"
12 #include "sleep.h"
13 #include "xil_printf.h"
14 #include "xil_io.h"
15
16 #define BASEADDR 0x43C00000
17
18 // NOTE(acol): Helper macros
19 #define ArrayCount(A) (sizeof(A) / sizeof(A[0]))
20 #define Signal(S) (signal){(u8*)(S), ArrayCount(S)}
21
22 // NOTE(acol): types
23 typedef uint8_t u8;
24 typedef uint32_t u32;
25
26 typedef struct signal {
27     u8 *Val;
28     u32 Length;
29 } signal;
30
31
32 #define RValid(R) ((R>> 19) & 0x1)
33 #define RValue(R) (R & ((1<<19) -1))
34
35 void RunFilter(signal X, u8 TapCount, u32 AReg, u32 BReg)
36 {
37     u32 Response = 0;
38
39     // NOTE(acol): Reset
40     Xil_Out32(AReg, (1<<9));
41     Xil_Out32(AReg, 0);
42
43     xil_printf("Y[n] = [ ");
44     for(u32 Idx = 0; Idx < X.Length + TapCount -1; Idx++)
45     {
46         // NOTE(acol): Data is the entire signal + enough zeros for the tail end
47         u8 Data = 0xff & ((Idx < X.Length)? X.Val[Idx] : 0);
48
49         // NOTE(acol): Send data and zero out
50         Xil_Out32(AReg, (1<<8) | Data);
51         Xil_Out32(AReg, 0);
52
53         // NOTE(acol): Read untill valid
54         do {
55             Response = Xil_In32(BReg);
56         } while(!RValid(Response));
57
58         xil_printf("%lu ", RValue(Response));
59     }
60     xil_printf("]\n\r");
61 }
62
63 int main()
64 {
65     init_platform();
66
67     u8 Dirac[] = {1, 0, 0, 0, 0, 0, 0, 0};
68     u8 Unit[] = {1, 1, 1, 1, 1, 1, 1, 1};
69     u8 Taps = 8;
70
71     RunFilter(Signal(Unit), Taps, BASEADDR, BASEADDR + 4);
72     RunFilter(Signal(Dirac), Taps, BASEADDR, BASEADDR + 4);
73
74     sleep(10);
75     cleanup_platform();
76     return 0;
77 }

```